UNIVERSIDAD NACIONAL DEL SUR
Departamento de Ciencias e Ingeniería de la Computación
Compiladores e Intérpretes — Segundo Cuatrimestre de 2026

Compilador de MiniJava

Etapa 4 — Analizador Semántico II

Árboles Sintácticos Abstractos y Chequeo de Sentencias

Agustín Paul
L.U. 147524

Informe de entrega

15 de octubre de 2026

Índice

1. Cómo compilar y ejecutar **1**

2. Los árboles sintácticos abstractos **1**

3. Logros **5**

1 Cómo compilar y ejecutar

Proyecto Maven con Java 25. Desde la carpeta `compilador/`:

```
$ mvn package          # compila, corre la suite y arma target/Compilador.jar
$ java -jar target/Compilador.jar programa1.java
Compilacion Exitosa

[SinErrores]
```

El fuente es el único argumento y puede tener cualquier extensión. Sin opciones corre esta etapa, que también se elige con `--etapa=4`. Las etapas anteriores siguen disponibles: `--etapa=1` corre sólo el léxico, `--etapa=2` sólo el sintáctico y `--etapa=3` el chequeo de declaraciones, sin mirar los cuerpos.

2 Los árboles sintácticos abstractos

El sintáctico arma el AST de cada cuerpo con las acciones de un esquema de traducción sobre su gramática LL(1), y lo cuelga del método o constructor de la tabla de símbolos. Los tipos de nodo salen de la gramática original de la cátedra: cada sentencia y cada expresión con significado es una clase. Si las declaraciones están bien y la tabla está consolidada, cada clase chequea el cuerpo de los constructores y métodos que ella declara: cada nodo controla lo suyo y delega en sus hijos. Una expresión devuelve su tipo, y calcula aparte si es un destino asignable o una llamada. El primer error semántico termina el análisis.

Los diagramas de clases (figuras 1 a 4) muestran todos los campos, constructores y métodos del código. Lo subrayado es estático y lo que está en cursiva, abstracto. Una referencia a otra clase dibujada es una relación, rotulada con el nombre del campo y su multiplicidad (`0..1` si puede faltar, `*` si es una colección, ninguna si es exactamente una), y una caja sin miembros se detalla en otro diagrama.



Figura 1: Las sentencias. `NodoBloque` es el cuerpo de cada método y constructor.

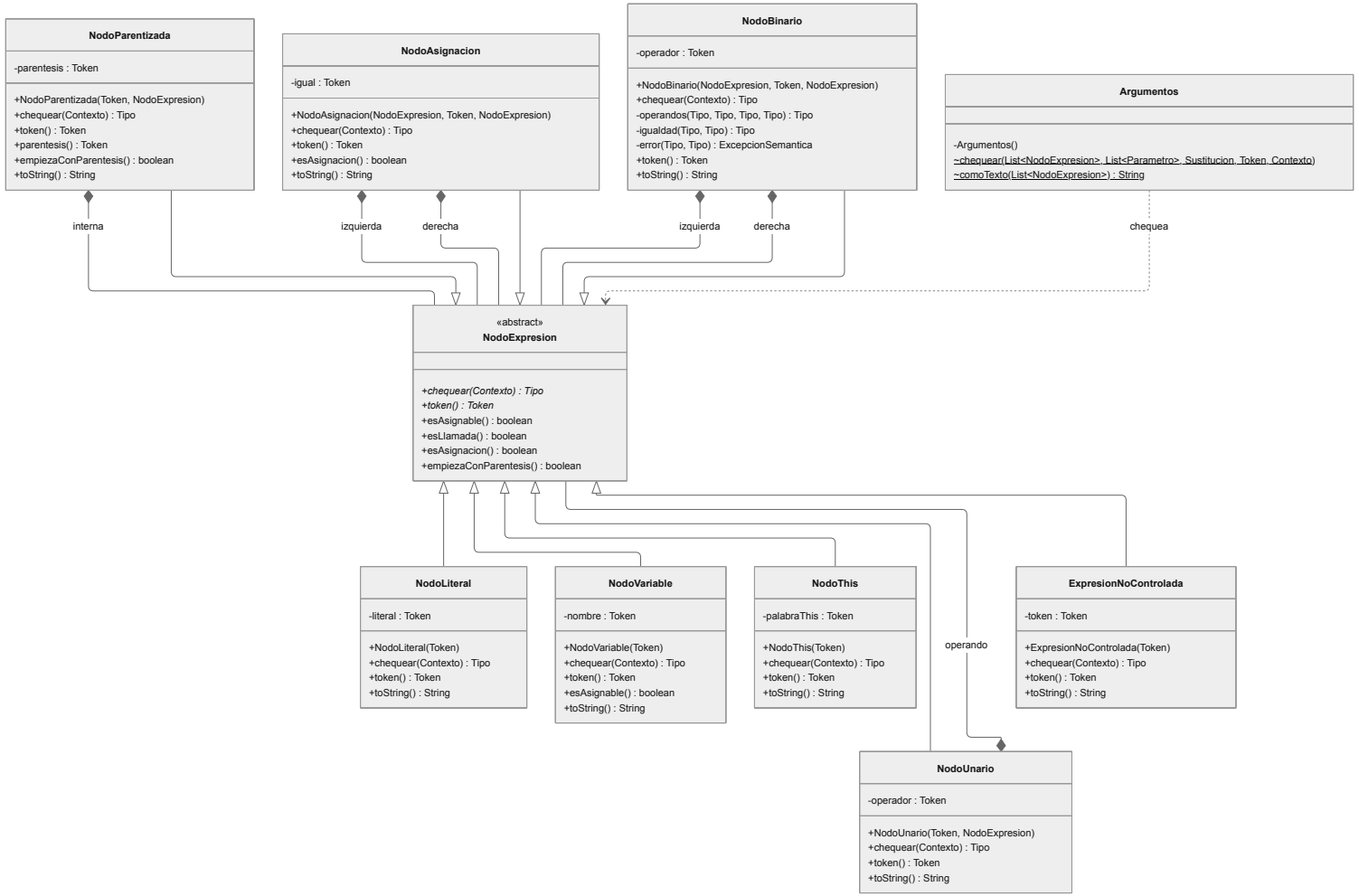


Figura 2: Las expresiones. `token()` es el token donde se reporta un error de la expresión entera.



Figura 3: Las llamadas, los accesos y las creaciones. Un encadenado guarda su receptor: `a.b.m()` es la llamada a `m` sobre el acceso a `b` sobre `a`.

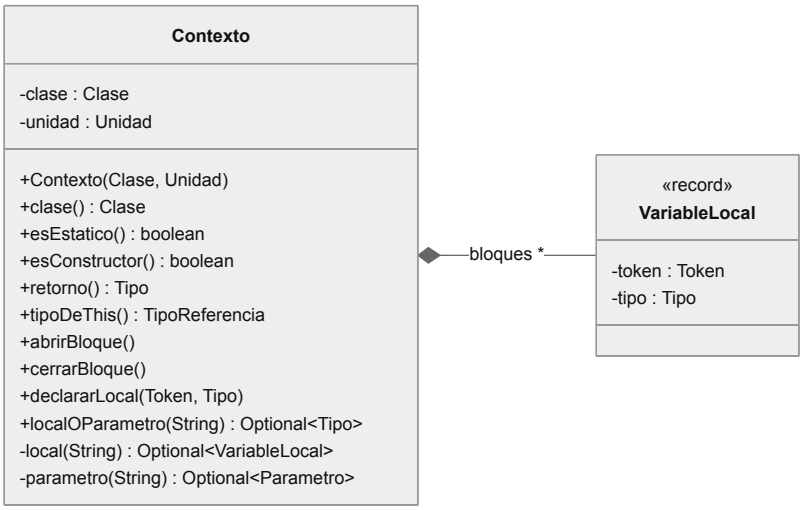


Figura 4: El contexto del chequeo de una unidad: la clase, la unidad y las variables locales visibles, un mapa por bloque abierto.

3 Logros

Se intentan los logros que no requieren código:

- **Imbatibilidad Semántica II**. Los casos de la cátedra dan el código exacto, y el corpus propio cubre cada control con casos con error y sin él.
- **Entrega Anticipada**.
- **Grandioso Laburo!**